

DOCKER 종합실습 1

작성일 : 2026-08-20 작성자 : 최승우

1. 소스코드

Nginx Web 컨테이너가 정적 화면과 /api 프록시를 제공하고, Flask WAS가 게시글 CRUD 및 PostgreSQL 연동을 담당한다.

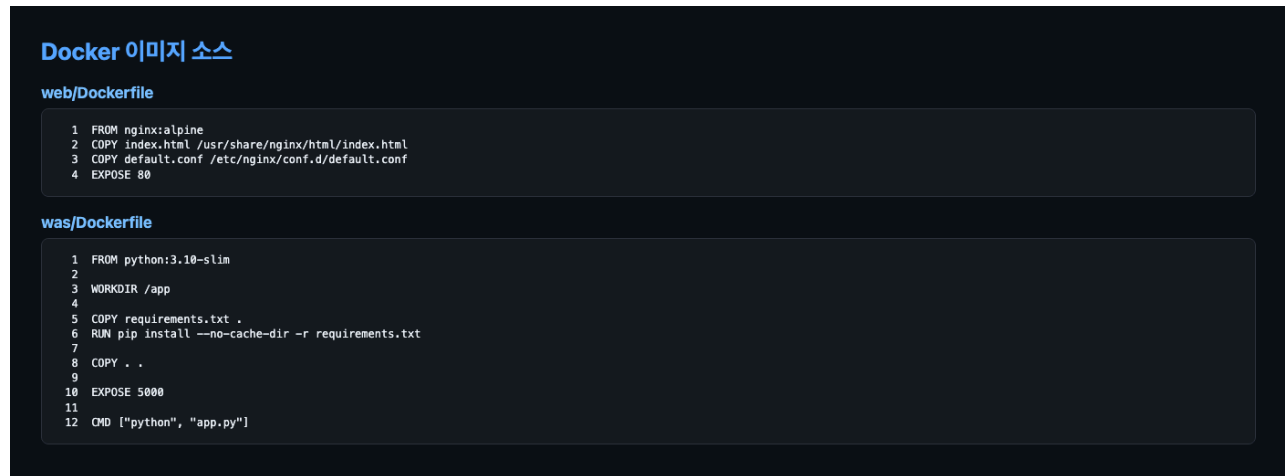


그림 1. web/Dockerfile 및 was/Dockerfile

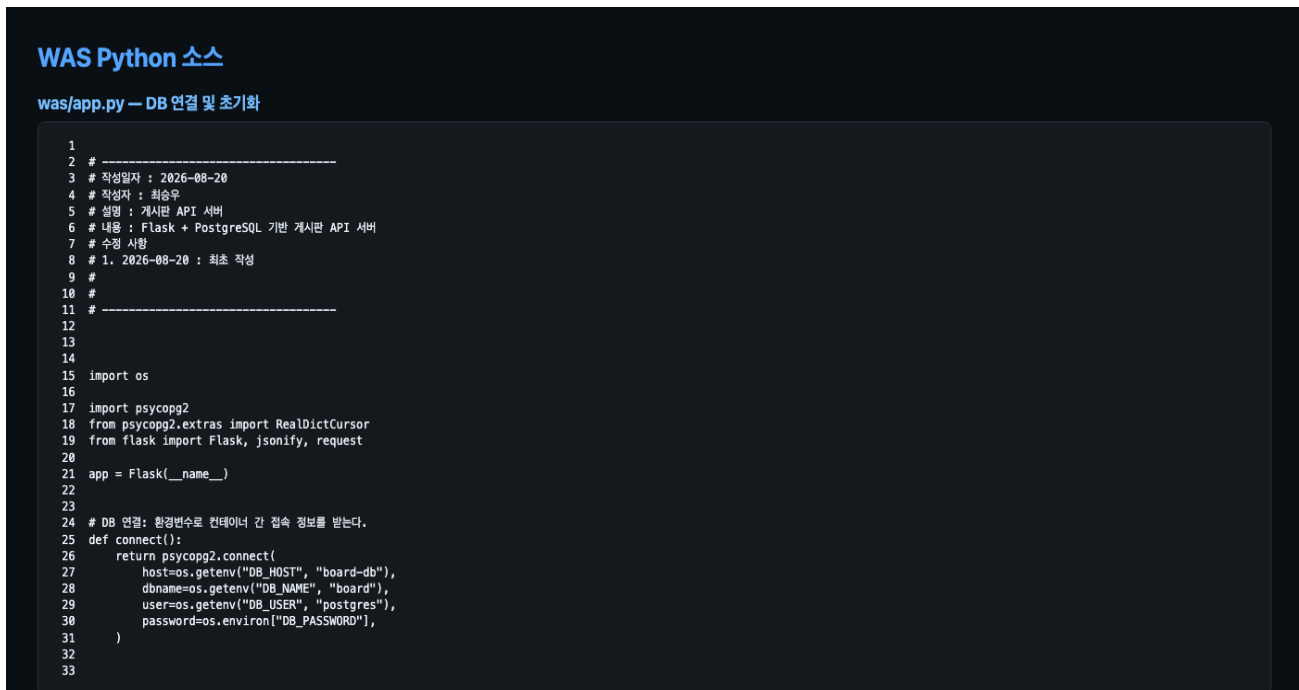


그림 2. was/app.py 핵심 코드 — DB 연결, 초기화 및 CRUD API

게시판 CRUD API

was/app.py · Flask + PostgreSQL

```
55 @app.get("/posts")
56 def get_posts():
57     with connect() as conn, conn.cursor(cursor_factory=RealDictCursor) as cursor:
58         cursor.execute("SELECT id, title, content, created_at FROM posts ORDER BY id DESC")
59         return jsonify(cursor.fetchall())
60
61
62 # 게시물 작성: 입력값을 검사하고 DB가 작성 시점을 자동 기록한다.
63 @app.post("/posts")
64 def create_post():
65     data = request.get_json(silent=True) or {}
66     if not data.get("title", "").strip() or not data.get("content", "").strip():
67         return {"error": "제목과 내용을 입력하세요."}, 400
68
69     with connect() as conn, conn.cursor(cursor_factory=RealDictCursor) as cursor:
70         cursor.execute(
71             "INSERT INTO posts (title, content) VALUES (%s, %s) RETURNING id, title, content, created_at",
72             (data["title"].strip(), data["content"].strip()),
73         )
74         return jsonify(cursor.fetchone()), 201
75
76
77 # 게시물 수정: 제목과 내용만 바꾸며 최초 작성 시점은 유지한다.
78 @app.put("/posts/<int:post_id>")
79 def update_post(post_id):
80     data = request.get_json(silent=True) or {}
81     if not data.get("title", "").strip() or not data.get("content", "").strip():
82         return {"error": "제목과 내용을 입력하세요."}, 400
83
84     with connect() as conn, conn.cursor(cursor_factory=RealDictCursor) as cursor:
85         cursor.execute(
86             "UPDATE posts SET title=%s, content=%s WHERE id=%s RETURNING id, title, content, created_at",
87             (data["title"].strip(), data["content"].strip(), post_id),
88         )
89         post = cursor.fetchone()
90         return (jsonify(post), 200) if post else ({ "error": "게시글을 찾을 수 없습니다." }, 404)
91
92
93 # 게시물 삭제: 없는 글은 404, 삭제 성공은 본문 없이 204를 반환한다.
94 @app.delete("/posts/<int:post_id>")
95 def delete_post(post_id):
96     with connect() as conn, conn.cursor() as cursor:
97         cursor.execute("DELETE FROM posts WHERE id=%s", (post_id,))
98         return ("", 204) if cursor.rowcount else ({ "error": "게시글을 찾을 수 없습니다." }, 404)
99
100
```

2. DB 생성 및 board 연동

PostgreSQL 17 컨테이너를 board-net에 연결하고 데이터베이스 board를 생성했다. 과거 `skala_db` 실습을 만들면서 호스트포트 5432를 사용중이기에, SQLTools과 dbeaver 호스트 포트는 5433을 사용해 연결했다. WAS는 Docker 내부 주소 board-db:5432로 접속한다.

DB 및 컨테이너 연동 확인

```
$ docker ps --filter name=board-
NAME        IMAGE        STATUS        PORTS
board-web   board-web:1.0 Up 11 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
board-was   board-was:1.0 Up 11 minutes 0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
board-db    postgres:17  Up 19 minutes 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp

$ docker network inspect board-net
board-web  -> board-was:5000
board-was  -> board-db:5432

$ psql -h localhost -p 5433 -U postgres -d board
id | title | created_at
---+-----+-----
 7 | 게시판 기능 점검 | 2026-08-20 06:59:59.522759+00
 6 | Docker 종합실습 완료 | 2026-08-20 06:59:59.505061+00
(2 rows)
```

그림 3. 컨테이너 실행 상태, 네트워크 경로 및 board.posts 조회 결과

연동 구성

- 사용자 브라우저 → board-web:80 (호스트 8080)
- board-web → board-was:5000 (/api 프록시)
- board-was → board-db:5432 (Docker 내부 통신)
- SQLTools → localhost:5433 / DB: board / User: postgres

DB 검증

posts 테이블은 id, title, content, created_at 컬럼으로 구성했다. created_at은 TIMESTAMPTZ와

CURRENT_TIMESTAMP 기본값을 사용해 글 작성 시 DB에서 자동 기록한다. 캡처 결과 두 게시글과 작성 시각이 저장된 것을 확인했다.

	123 id	A-Z title	A-Z content	created_at
1	8	test	입력값	2026-08-20 16:05:45.192 +0900
2	9	수정사항	수정	2026-08-20 16:06:16.770 +0900

3. 실행결과 및 코드 리뷰

게시판

제목

내용

작성

글 목록

게시판 기능 점검

2026. 8. 20. 오후 3:59:59

글 작성일 자동 저장과 수정·삭제 기능이 정상 동작합니다.

수정 삭제

Docker 종합실습 완료

2026. 8. 20. 오후 3:59:59

Web, WAS, PostgreSQL 컨테이너 연동 및 CRUD 기능을 확인했습니다.

수정 삭제

그림 4. 게시글 목록, 작성일, 수정·삭제 버튼이 표시된 실행 화면

실행결과

localhost:8080에서 글 목록 조회, 신규 작성, 수정, 삭제를 확인했다. 작성일은 DB 저장값을 한국어 날짜 형식으로 변환해 표시하며, 컨테이너 재시작 후에도 게시글이 유지된다.

코드 리뷰 (권주현)

Q. Dockerfile의 FROM에는 왜 이 베이스 이미지를 사용했어요?

A. Web은 정적 파일과 프록시만 필요해 nginx:alpine을, WAS는 Flask 실행에 필요한 python:3.10-slim을 사용했습니다. 이번 실습에서 제시된 예제이지만 간단한 웹 프로젝트 기능 구현에 적합하고 크기가 작은 이미지라는 이유가 큰 것으로 보입니다.

Q. 불필요한 파일까지 COPY하지 않았나요?

A. Web은 필요한 두 파일만 복사합니다. WAS의 COPY ..는 __pycache__까지 포함할 수 있어 부분 개선이 필요하며, .dockerignore 추가가 좋습니다.

Q. EXPOSE 포트와 실제 -p 옵션의 포트는 왜 다르게 보이나요?

A. EXPOSE는 내부 포트입니다. Web은 80/-p 8080:80, WAS는 5000/-p 5001:5000으로 컨테이너 내부 포트가 정확히 일치합니다.

Q. 환경 변수 이름이 코드와 실행 명령에서 일치하나요?

A. 일치합니다. app.py와 docker run이 DB_HOST, DB_NAME, DB_USER, DB_PASSWORD를 동일하게 사용해 board-db에 접속합니다.

Q. 컨테이너 이름이 서로 겹치지 않도록 했나요?

A. board-web, board-was, board-db로 역할별 고유 이름을 사용했습니다. 이 이름은 Docker DNS 주소로도 사용되어 연결 관계가 분명합니다.

결론: 요구 기능과 Web-WAS-DB 연동이 모두 정상 동작하며, COPY 범위만 보완하면 실습 목적에 맞춰서 잘 작성했다

주현 : Web, WAS, DB의 역할이 명확하고 API 구조가 단순해 이해하기 쉽다. SQL 파라미터 바인딩과 textContent를 사용해 기본 보안도 고려했다. 다만 운영 환경에서는 요청별 DB 연결을 연결 풀로 개선하고, init_db()의 스키마 변경을 별도 마이그레이션으로 관리하는 것이 좋겠다.